

CASE STUDY

CampusBite: A Cloud-Based Canteen Pre-Order and Token System

Deployed on PaaS (Render) with a managed database and object storage (Supabase), using free-tier cloud services

Subject	Cloud Computing
Topics covered	Cloud service models (IaaS / PaaS / SaaS), public cloud deployment, cloud storage and databases, cloud security, scalability and elasticity, pay-as-you-go cost model
Name / Reg. No.	Yashodhan Singh Rathore / RA2311003010854

1. Problem Analysis and Cloud Design

1.1 Problem identification

During lunch breaks, a college canteen receives most of its orders in a window of about 30 minutes. Students stand in long queues, orders are noted on paper, items run out without warning, and the staff cannot predict demand. Existing options are either manual (no record, no tracking) or generic food-delivery apps that are not built for a single campus and charge commissions.

CampusBite lets a student log in, view the live menu with photos, place an order in advance, and receive a token number. Canteen staff see the incoming order queue and mark orders as Preparing or Ready. The system must stay available during the lunch rush, protect student data, and cost almost nothing to run.

1.2 Requirements

Functional requirements	Non-functional requirements
Student registration and login Browse menu with images and availability Place order and get a unique token Track order status (Placed / Preparing / Ready) Staff dashboard to update menu and order status	Availability during peak load (lunch rush) Security: encrypted traffic, hashed passwords, role-based access Low latency (page response under 1 second when the app is awake) Low cost (free tier for the prototype) Easy deployment from source control with no server administration

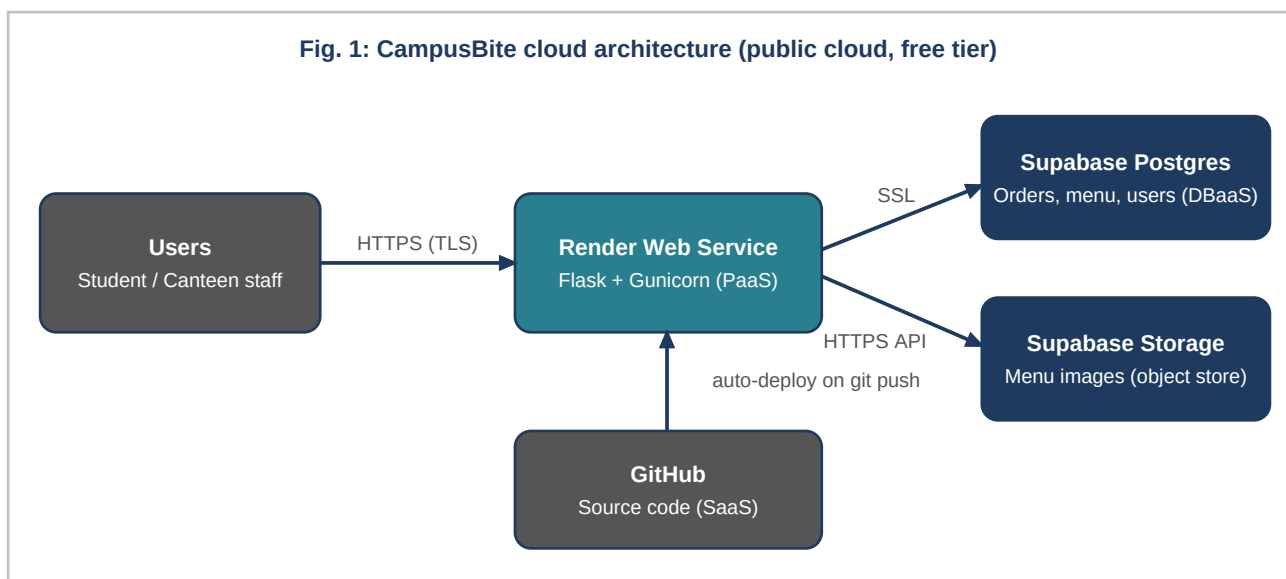
1.3 Selection of service and deployment models

Component	Service model	Reason for the choice
Render Web Service (Flask app)	PaaS	Provider manages the OS, runtime, TLS certificate and deployment. We only supply code and a start command. IaaS (a raw VM) would need manual patching, Nginx setup and firewall management, which is unnecessary for this app.
Supabase PostgreSQL	DBaaS (managed PaaS)	Managed relational database with backups and SSL. Orders and users are relational data, so SQL fits better than NoSQL.
Supabase Storage	Cloud object storage	Stores menu images outside the app server, so the web service stays stateless and can be restarted or scaled freely.
GitHub	SaaS	Hosted version control that also acts as the deployment trigger.

Deployment model: public cloud. The services are shared multi-tenant platforms accessed over the internet, which suits a campus prototype with no need for private infrastructure. **Why not IaaS?** A VM gives more control but adds administration work and a cost that does not pay off for a small, spiky workload.

1.4 Cloud architecture

The application is a stateless web service. The browser talks to the web service over HTTPS, the web service reads and writes orders in a managed PostgreSQL database, and menu images are stored in an object storage bucket. The code lives in GitHub, and every push triggers an automatic redeploy.



2. Implementation and Technical Execution

2.1 Technology stack

Layer	Technology
Backend	Python 3, Flask, Gunicorn (production WSGI server), psycopg2 for PostgreSQL, bcrypt/Werkzeug for password hashing
Frontend	HTML, CSS and a little JavaScript (Jinja2 templates served by Flask)
Database	Supabase PostgreSQL (free tier)
File storage	Supabase Storage public bucket named menu-images
Hosting	Render Web Service (free tier) connected to a GitHub repository

2.2 Deployment steps

- 1 **Source control:** Push the Flask project (app.py, requirements.txt, templates/, static/) to a GitHub repository.
- 2 **Database:** Create a Supabase project. Create the tables in Section 2.3 using the SQL editor. Copy the connection string (with SSL) into a safe note.
- 3 **Storage:** Create a bucket named menu-images, upload the menu photos, and store each public image URL in the menu_items table.
- 4 **Web service:** On Render, choose New > Web Service, connect the GitHub repo, set Build Command to `pip install -r requirements.txt` and Start Command to `gunicorn app:app`.
- 5 **Secrets:** Add DATABASE_URL and SECRET_KEY as environment variables in the Render dashboard, so no credential appears in the code or on GitHub.
- 6 **Go live:** Render builds the app and issues a public HTTPS URL of the form `https://campusbite.onrender.com`. Any later git push redeploys automatically.

2.3 Database design

Table	Main columns
users	id (PK), name, email (unique), password_hash, role (student / staff)
menu_items	id (PK), name, price, image_url, available (boolean)
orders	id (PK), user_id (FK), token_no, status (Placed / Preparing / Ready), created_at
order_items	id (PK), order_id (FK), menu_item_id (FK), quantity

2.4 Networking

- The user reaches the app through a public HTTPS endpoint on the provider's domain. DNS and the TLS certificate are managed by Render.
- The app connects to the database over SSL using the connection string held in an environment variable. Images are loaded directly by the browser from the storage URL, which keeps image traffic off the app server.
- All traffic is request/response over standard HTTPS (port 443). The app server is the only entry point for data operations.

2.5 Security measures

Threat	Control implemented
Eavesdropping	HTTPS/TLS for browser to app, SSL for app to database
Password theft	Passwords stored only as salted hashes (never plain text)
Unauthorised actions	Role-based access: only users with role = staff can change menu and order status; sessions use a signed SECRET_KEY
SQL injection	Parameterised queries for every database call
Credential leakage	Secrets in environment variables, .env listed in .gitignore

2.6 Working demonstration flow

(1) Register and log in as a student. (2) Open the menu and add items to the cart. (3) Place the order and receive a token number. (4) Log in as staff in another browser, see the new order in the queue and mark it Ready. (5) The student's order page now shows Ready. (6) Show the live HTTPS link, the Supabase tables updating, and the Render deploy log.

3. Results and Technical Evaluation

3.1 Test method

The deployed application was load tested from a laptop to simulate the lunch rush. Apache Bench (ab) sends many requests with a chosen number of simultaneous users to the menu page, which is the most visited page.

```
ab -n 500 -c 10 https://campusbite.onrender.com/menu
ab -n 500 -c 50 https://campusbite.onrender.com/menu
```

3.2 Measured results

Test	Total requests	Concurrent users	Requests/sec	Avg latency (ms)	Failed requests
Light load	500	10	[fill]	[fill]	[fill]
Lunch-rush load	500	50	[fill]	[fill]	[fill]
First request after idle (cold start)	1	1	-	[fill] s	-

Note for the author: run the two ab commands after deployment and replace each [fill] with the value Apache Bench prints (Requests per second, Time per request (mean), Failed requests). Also note the cold-start time from the Render free tier by opening the site after 15 minutes of inactivity. Attach the screenshots in the appendix.

3.3 Scalability analysis

- **Stateless design:** sessions are signed cookies and all data is in the database and object storage, so any number of identical app instances can serve any request. This is what makes horizontal scaling possible.
- **Vertical scaling:** moving the Render service to a larger paid instance adds CPU and memory with no code change.
- **Horizontal scaling:** paid Render plans can run multiple instances behind the platform's load balancer, and Supabase can add connection pooling for more simultaneous database connections.
- **Free-tier behaviour:** the free web service sleeps after about 15 minutes of no traffic and takes several seconds to wake on the next request, and resources are limited. This is acceptable for a prototype and is a known trade-off, solved by a paid always-on instance in production.

3.4 Cost analysis

Option	Components	Approx. monthly cost
This project (free tier)	Render free web service + Supabase free + GitHub free	USD 0
Small production setup (PaaS)	Render paid starter instance + Supabase free or low tier	About USD 7 to 32
Equivalent IaaS setup on AWS	EC2 t3.micro + RDS db.t3.micro + S3 + data transfer	About USD 20 to 30 plus admin effort

Prices are approximate list prices and change over time; confirm them on the provider pricing pages or the AWS Pricing Calculator before quoting them. The point of the comparison is that PaaS removes both the server administration effort and most of the fixed cost for a small workload.

3.5 Technical justification

- PaaS was chosen because the team needs to deliver features, not manage servers. Patching, TLS and deployment are handled by the provider.
- A managed relational database gives backups, SSL and ACID transactions, which matter for orders and token numbers that must never be duplicated.
- Separating images into object storage keeps the web tier stateless and light, so it scales and restarts easily.
- Source control triggered deployment gives repeatable releases and a quick rollback path.

3.6 Limitations and future work

The prototype runs in a single region and a single instance, so there is no failover; free-tier limits apply; and there is no payment gateway. Future work includes adding a CDN for images, an always-on paid instance with auto-scaling, email or SMS notification when an order is ready, online payment, and a monitoring dashboard with alerts.

4. Conclusion

CampusBite shows how a real campus problem can be solved with cloud services in a few hours: a PaaS web tier, a managed database, object storage and automated deployment from GitHub. The design is secure by default, costs nothing at prototype scale, and can scale up by changing the plan rather than rewriting the code. It demonstrates the practical difference between IaaS, PaaS and SaaS and why a managed service is often the right choice for small, spiky workloads.